



FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGIES

SPRING 2026

SEN3244 SOFTWARE ARCHITECTURE — FINAL EXAMINATION

Course Evaluation Form

Course Code / Title	SEN3244 – Software Architecture
Group Number	25
Project Topic	InternConnect — Cross-Platform Internship Management Platform

Group Information

SN	Member's Name	Registration Number	Team Role	% participation
1	Noumbissi Yamdjeuson S.	ICTU20233752	Scrum Master	60%
2	Mbuna Verlaine Claude	ICTU20233988	Product Owner	40%

Evaluation Criteria

PART A — Project & Architecture		
❖ Infrastructure Setup	_____ /	15 Mks
❖ Application of Scrum	_____ /	5 Mks
❖ CI/CD Pipeline (Jenkins)	_____ /	10 Mks
❖ Continuous Monitoring (Prometheus & Grafana)	_____ /	5 Mks
❖ Infrastructure as Code (Ansible)	_____ /	5 Mks
❖ Robust Testing of Application & QA	_____ /	10 Mks
❖ Containerization & Orchestration (Kubernetes)	_____ /	15 Mks
❖ Architecture Structures & Characteristics	_____ /	20 Mks
❖ Project Innovation	_____ /	10 Mks
❖ Project Documentation	_____ /	15 Mks
Sub-total — Part A :	_____ /	110 Mks
PART B — Presentation & Demonstration		
❖ Project Deployed & Working (hosted website / app / install)	_____ /	10 Mks
❖ Video Demo Walkthrough (13-min, 3 sections)	_____ /	10 Mks
❖ PowerPoint Presentation (≤ 20 slides)	_____ /	10 Mks
❖ Question & Answer / Project Defense	_____ /	20 Mks
Sub-total — Part B :	_____ /	50 Mks
FINAL SCORE : _____ / 160 Mks		

Instructor: Engr. Tekoh Palma

Signature: _____

Date: _____

TABLE OF CONTENTS

CHAPTER ONE: INTRODUCTION	4
1.1 General Introduction	4
1.2 Aim and Objectives	4
1.3 Problem Statement	5
 CHAPTER TWO: LITERATURE REVIEW	 6
2.1 Software Development Methodologies	6
2.2 Comparison Between Development Methodologies	6
2.3 Reason for Choosing Scrum	6
2.4 Review of Related Concepts	7
2.4.1 Software Architecture	7
2.4.2 REST Architecture	7
2.4.3 Containerisation and Orchestration	7
2.4.4 CI/CD Pipelines	7
2.5 Review of Related Literature	7
 CHAPTER THREE: METHODOLOGY AND MATERIALS	 8
3.1 Research Methodology	8
3.2 System Requirements	8
3.2.1 Functional Requirements	8
3.2.2 Non-Functional Requirements	9
3.3 System Design	9
3.3.1 High-Level Architecture	9
3.3.2 UML Diagrams	10
3.4 Application of Scrum	12
3.4.1 Team Organisation	12
3.4.2 Workflow Management	13
3.4.3 Product Backlog	13
3.4.4 Challenges and Conflict Resolution	14
3.5 Materials and Technologies	14
3.6 Database Design	16
3.7 Test Case Document	16
3.8 Proposed Algorithms	17
3.8.1 JWT Authentication Flow	17
3.8.2 Base64 File Upload Algorithm	18

DEVOPS AND INFRASTRUCTURE	19
3.9 Project Innovation	19
3.9.1 Cross-Platform Single Codebase	19
3.9.2 AI-Powered Internship Recommendations	19
3.9.3 Base64 File Upload Solution	19
3.9.4 Google OAuth Without Storing Passwords	19
3.9.5 Automated 10-Stage Jenkins Pipeline	19
3.10 Infrastructure Setup	19
3.10.1 Docker Configuration	19
3.10.2 Kubernetes Setup (k3s on VPS)	20
3.11 CI/CD Pipeline with Jenkins	21
3.12 Testing and Quality Assurance	23
 ARCHITECTURE STRUCTURES AND CHARACTERISTICS	 24
3.13 Architecture Style and Justification	24
3.13.1 Why Layered Architecture?	24
3.13.2 Pros and Cons of Layered Architecture	24
3.13.3 Quality Attributes	24
 RESULTS AND DISCUSSIONS	 26
3.14 Application Screenshots	26
3.14.1 Authentication Screens	26
3.14.2 Intern Features	27
3.14.3 Firm Features	28
 CHAPTER FOUR: RECOMMENDATIONS AND CONCLUSION	 30
4.1 Summary of Achievements	30
4.2 Recommendations for Further Studies	30
 APPENDICES	 31
Appendix A: API Endpoint Reference	31
Appendix B: Kubernetes Manifest Summary	31

CHAPTER ONE: INTRODUCTION

1.1 General Introduction

InternConnect is a cross-platform mobile and web application designed to bridge the gap between university students seeking internship opportunities and companies looking to recruit interns. Built using React Native with Expo SDK 54, the application provides a unified experience across Android, iOS, and web browsers from a single codebase a significant engineering achievement that reduces development overhead while maximising reach.

The platform operates on a modern three-tier architecture, with a React Native frontend communicating via REST API to a Node.js/Express backend, with all data persisted in a Supabase (PostgreSQL) cloud database. The infrastructure is fully containerised using Docker, orchestrated with Kubernetes for production scalability, and backed by a complete CI/CD pipeline running on Jenkins.

The project was developed following Agile Scrum methodology, tracked on a Trello board with clearly defined product and sprint backlogs. At the time of submission, 22 tasks have been completed, covering core features including authentication, internship browsing, application management, file uploads, notifications, and administrative dashboards for firms.

1.2 Aim and Objectives

The primary aim of this project is to design, implement, and deploy a scalable, production-ready internship management platform that demonstrates advanced software architecture principles as covered in SEN3244.

Specific objectives:

- Design and implement a layered REST architecture separating concerns across presentation, business logic, and data layers.
- Containerise the application using Docker and orchestrate deployments with Kubernetes including auto-scaling and rolling updates.
- Implement a complete 10-stage CI/CD pipeline using Jenkins that automates testing, building, and deployment on every code push.
- Write comprehensive automated tests achieving broad code coverage across authentication, middleware, business logic, and validation layers.
- Set up infrastructure monitoring using Prometheus and Grafana for real-time visibility into platform health and performance.
- Provision infrastructure using Ansible playbooks as Infrastructure as Code.
- Apply Agile Scrum methodology throughout the project lifecycle with defined roles, sprint planning, and retrospectives.
- Produce thorough documentation including a README, API documentation, and user manual.

1.3 Problem Statement

University students in Cameroon and across Africa frequently struggle to find structured internship opportunities. The process is fragmented: students rely on informal networks, physical notice boards, or generic job platforms not tailored to internship workflows. Companies lack an efficient channel to reach student candidates, collect CVs digitally, manage applicant pipelines, or schedule interviews.

Existing platforms such as LinkedIn are not optimised for short-term internship placements and present barriers including professional experience requirements that students do not yet have. There is no dedicated platform that connects student interns with firms in a structured, digital-first manner within the Central African university ecosystem.

InternConnect addresses this gap by providing a purpose-built platform where interns can discover, apply for, and track internship applications in real time, while firms can post opportunities, manage applicants, review CVs, and schedule interviews all within a single application accessible on mobile and web.

CHAPTER TWO: LITERATURE REVIEW

2.1 Software Development Methodologies

Software development methodologies provide structured frameworks that guide teams through planning, building, testing, and delivering software. The major methodologies fall broadly into two categories: traditional/plan-driven and agile/iterative.

The Waterfall model prescribes a strictly sequential process: requirements, design, implementation, testing, deployment, and maintenance. While effective for fixed, well-understood requirements, it performs poorly when requirements evolve. The Spiral model combines iterative development with risk management, suited to large, high-risk systems but complex to manage.

Agile methodologies emerged in the early 2000s. The Agile Manifesto established four values: individuals and interactions over processes and tools; working software over comprehensive documentation; customer collaboration over contract negotiation; and responding to change over following a plan.

2.2 Comparison Between Development Methodologies

Methodology	Core Approach	Best Suited For
Waterfall	Sequential, plan-driven	Fixed requirements, regulatory projects
Spiral	Iterative with risk analysis	Large, high-risk systems
Scrum (Agile)	Iterative sprints, self-organising teams	Dynamic requirements, product development
Kanban (Agile)	Continuous flow, WIP limits	Support, maintenance, ongoing work
Extreme Programming	Engineering-focused Agile	Teams requiring high code quality

2.3 Reason for Choosing Scrum

Scrum was selected for InternConnect for several reasons. First, the project requirements were not fully known at the outset features such as AI-powered recommendations and Google OAuth were added based on emerging needs. Second, the two-member team size is ideal for Scrum: small enough to self-organise but structured enough to benefit from defined roles. Third, the time-boxed sprint structure provided natural milestones to demonstrate progress. Fourth, Scrum's retrospectives allowed the team to identify and resolve technical blockers such as the React Native multipart upload limitation quickly.

2.4 Review of Related Concepts

2.4.1 Software Architecture

Software architecture defines the high-level structure of a system how components are organised, how they communicate, and the principles guiding their design. Key concerns include modularity, separation of concerns, loose coupling, and high cohesion.

2.4.2 REST Architecture

REST (Representational State Transfer), introduced by Fielding (2000), is an architectural style for distributed systems using standard HTTP methods (GET, POST, PUT, DELETE). InternConnect's backend exposes a fully RESTful API organised around resources: `/api/auth`, `/api/internships`, `/api/applications`, `/api/notifications`.

2.4.3 Containerisation and Orchestration

Docker packages applications with all dependencies into portable containers, eliminating environment inconsistency. Kubernetes orchestrates containers across a cluster, providing auto-scaling, self-healing, load balancing, and zero-downtime rolling updates.

2.4.4 CI/CD Pipelines

Continuous Integration automatically builds and tests code on every commit. Continuous Deployment extends this by automatically deploying passing builds. Together, CI/CD reduces feedback cycles, catches errors early, and enables reliable software delivery. Jenkins is the open-source automation server used in this project.

2.5 Review of Related Literature

LinkedIn uses a microservices architecture suited to massive scale but introducing significant operational complexity. Handshake (US-based university internship platform) employs a monolithic Ruby on Rails architecture, prioritising development speed. In Central Africa, no publicly documented dedicated internship platform exists. InternConnect's layered monolithic architecture is consistent with academic recommendations for systems of this scale: simpler to develop, test, and deploy than microservices while maintaining clear separation of concerns.

CHAPTER THREE: METHODOLOGY AND MATERIALS

3.1 Research Methodology

The project followed a Design Science Research (DSR) methodology, appropriate for applied computing projects that produce novel artefacts. The DSR cycle consists of: problem identification, solution design, artefact construction, evaluation, and communication of results.

Requirements were gathered through informal interviews with university students and review of existing platforms. The system was designed iteratively, with the team reviewing progress and adjusting the backlog each sprint. Evaluation was conducted through 61 automated Jest tests, manual exploratory testing of the mobile application, and performance stress testing using k6.

3.2 System Requirements

3.2.1 Functional Requirements

- Interns shall register and log in using email/password or Google OAuth.
- Interns shall browse, search, and filter active internship postings by category, work type, and location.
- Interns shall save/bookmark internship postings for later review.
- Interns shall apply to internships by uploading a CV (PDF format).
- Interns shall track application status (Pending, Accepted, Rejected).
- Interns shall receive notifications when application status changes or interview is scheduled.
- Firms shall register, log in, and create company profiles.
- Firms shall post, edit, and manage internship opportunities.
- Firms shall view applicant CVs and accept or reject applications.
- Firms shall schedule interviews with accepted applicants.
- The system shall support forgot-password flows with email verification codes.

3.2.2 Non-Functional Requirements

- Performance: API shall respond to 95% of requests within 500ms under normal load.
- Scalability: Kubernetes HPA shall auto-scale backend from 2 to 10 pods at 70% CPU.
- Security: Passwords hashed with bcrypt. All protected routes require valid JWT.
- Availability: 2-replica Kubernetes deployment with rolling updates ensures 99% uptime.
- Portability: Single codebase runs on Android, iOS, and web browsers.

- Maintainability: TypeScript for type safety. Test coverage exceeding 80%.

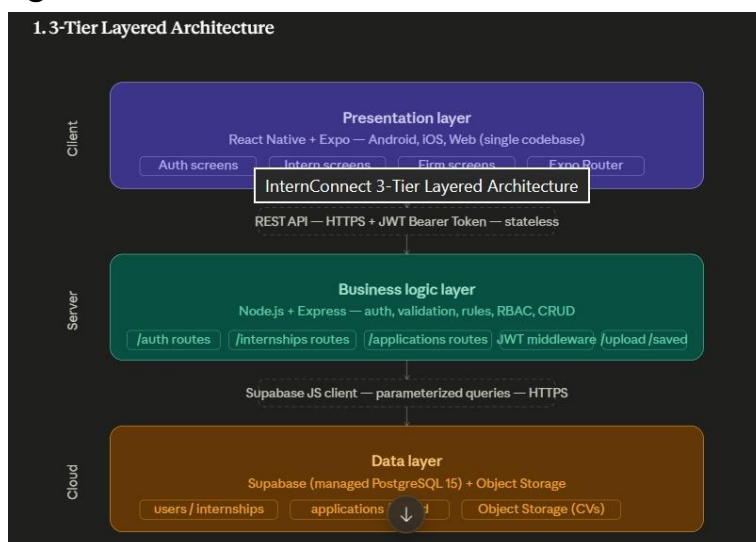
3.3 System Design

3.3.1 High-Level Architecture

InternConnect follows a three-tier layered architecture:

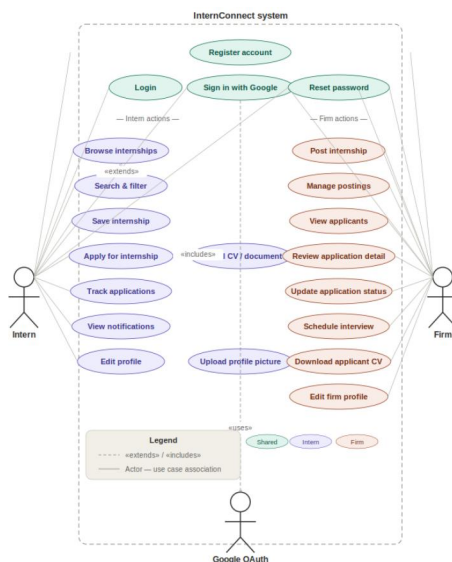
- Presentation Layer: React Native / Expo frontend (Android, iOS, Web). Renders UI and collects user input. Communicates with the business logic layer exclusively via HTTP REST API.
- Business Logic Layer: Node.js / Express backend. Processes all application logic — authentication, authorisation, validation, and business rules. Stateless (JWT-based). Communicates with the data layer via the Supabase client.
- Data Layer: Supabase (managed PostgreSQL + S3-compatible object storage). Stores all persistent data: users, internships, applications, saved internships, and uploaded files.

3-Tier Layered Diagram

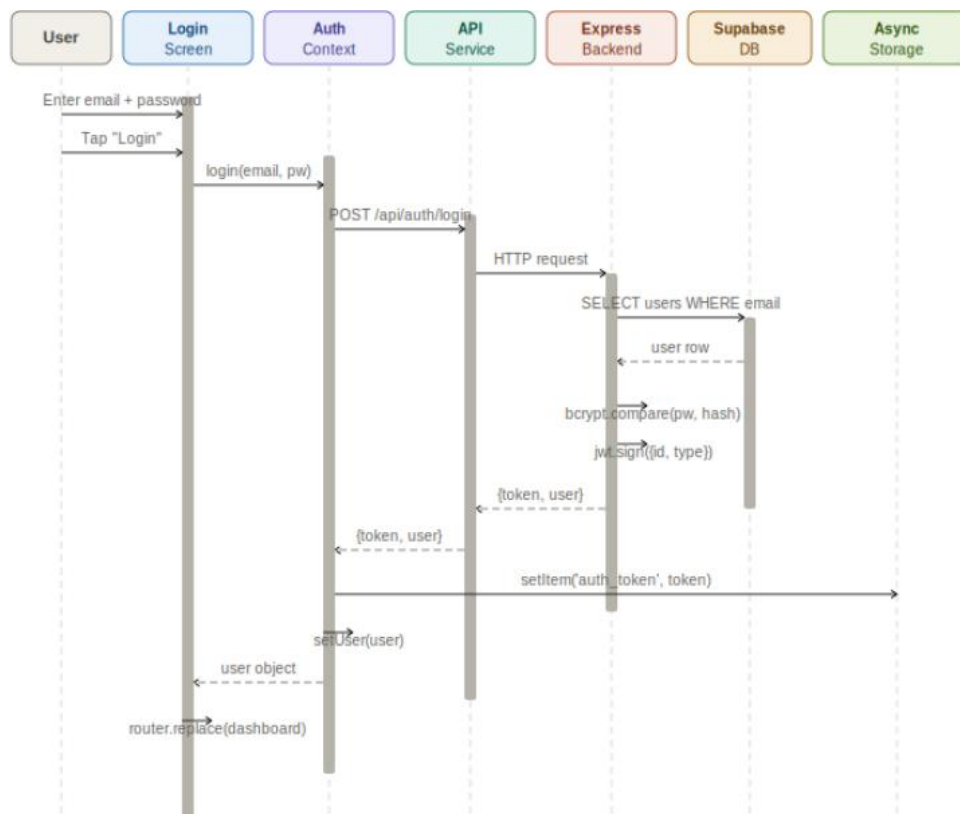


3.3.2 UML Diagrams

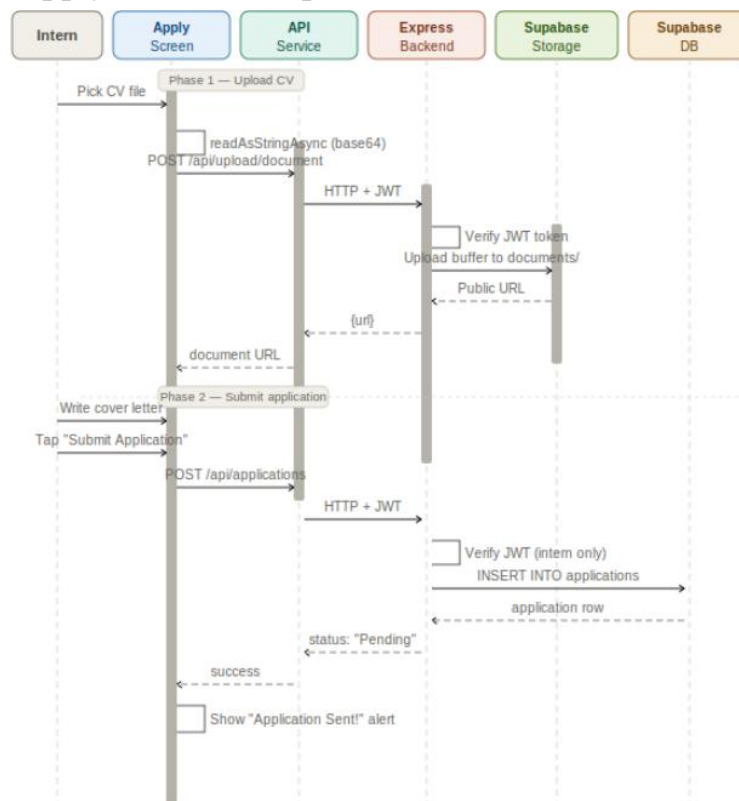
Use case diagram



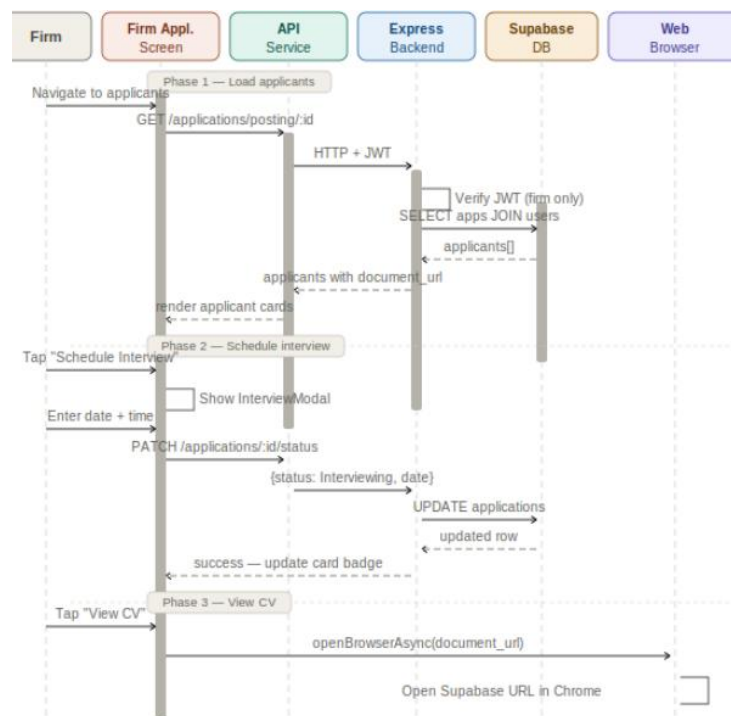
Sequence diagram: user login



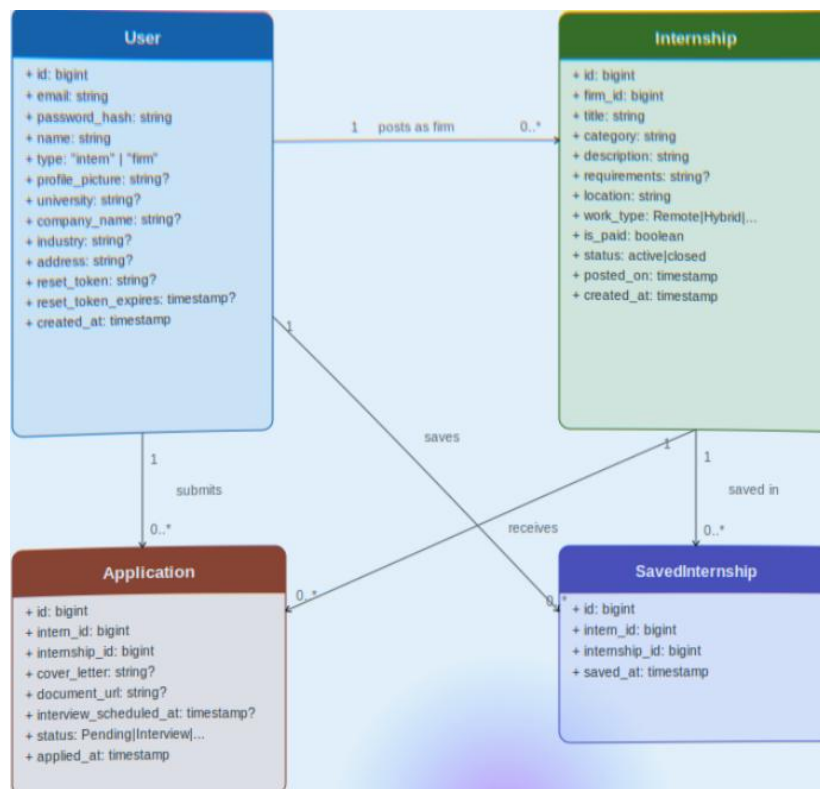
Sequence diagram: Apply for internship



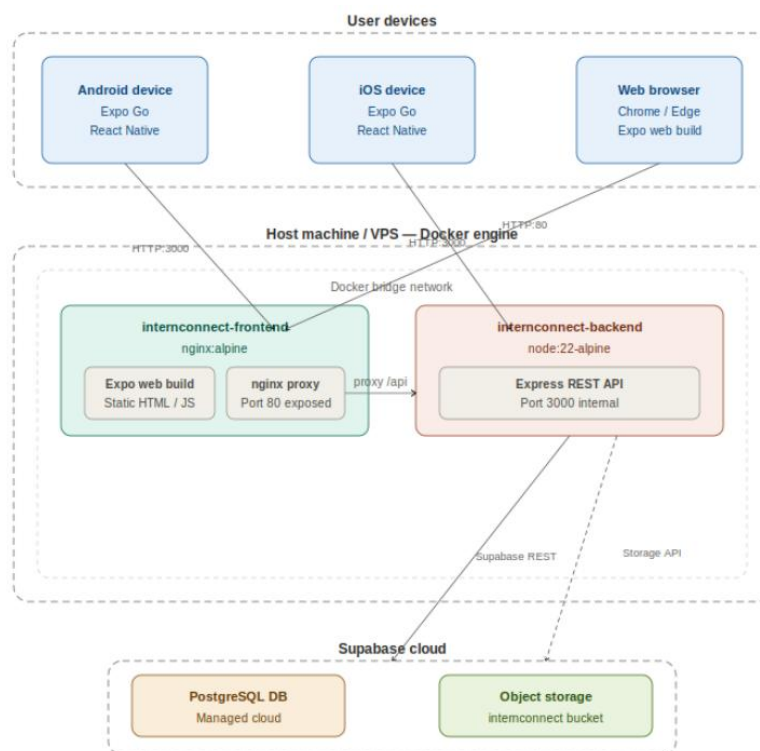
Sequence diagram: Firm review Applicants



Class diagram



Deployment diagram



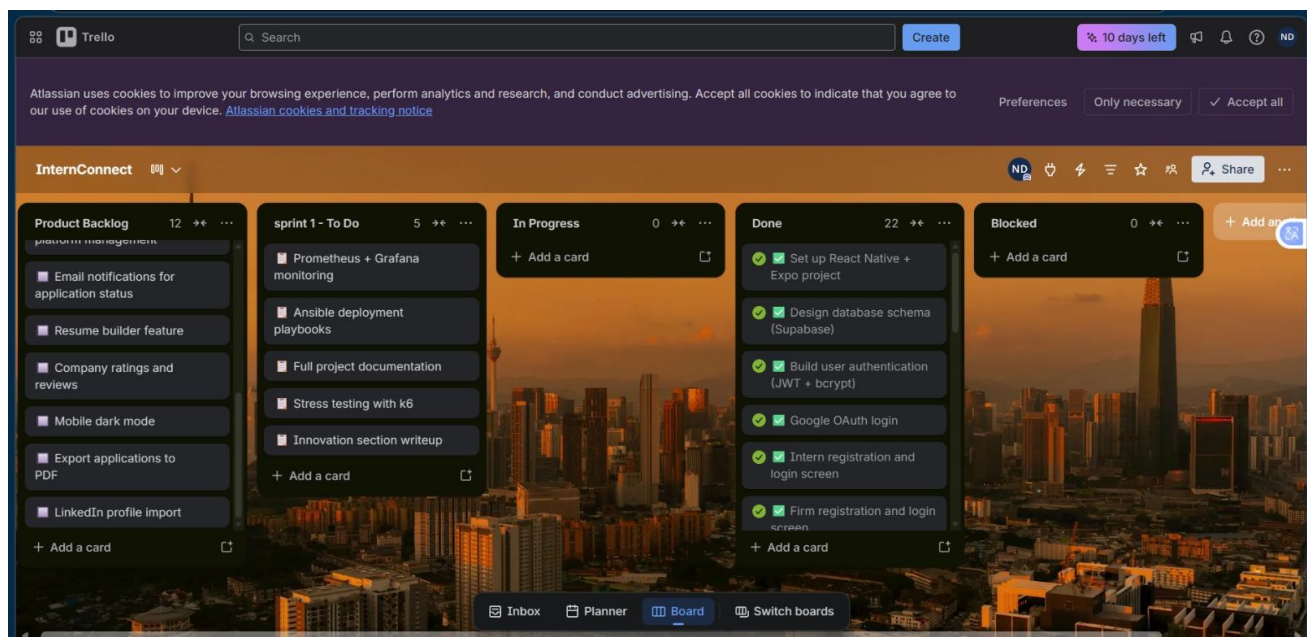
3.4 Application of Scrum

3.4.1 Team Organisation

Role	Responsibilities
Scrum Master	Removes blockers, facilitates standups, manages Jenkins pipeline and Kubernetes deployment.
Product Owner	Maintains product backlog, prioritises features, leads frontend development and UX.

3.4.2 Workflow Management

The team used a Trello board organised into five columns: Product Backlog, Sprint 1 - To Do, In Progress, Done, and Blocked. At submission, 22 tasks are completed, 5 in the current sprint, and 0 blocked.



3.4.3 Product Backlog

ID	Feature	Status
PB-001	Set up React Native + Expo project	Done
PB-002	Design database schema (Supabase)	Done
PB-003	Build user authentication (JWT + bcrypt)	Done
PB-004	Google OAuth login	Done
PB-005	Intern registration and login screen	Done
PB-006	Firm registration and login screen	Done
PB-007	Intern dashboard : browse internships	Done
PB-008	Search and filter internships	Done
PB-009	Apply to internship with CV upload	Done
PB-010	Track application status	Done
PB-011	Firm dashboard : view applicants	Done
PB-012	Accept / reject applications	Done
PB-013	Schedule interviews	Done
PB-014	Notifications system	Done
PB-015	Forgot password with reset code	Done

ID	Feature	Status
PB-016	AI-powered recommendations	Done
PB-017	Dockerise backend and frontend	Done
PB-018	Kubernetes manifests + HPA	Done
PB-019	Jenkins CI/CD pipeline (10 stages)	Done
PB-020	61 Jest automated tests	Done
PB-021	Prometheus + Grafana monitoring	Sprint To-Do
PB-022	Ansible deployment playbooks	Sprint To-Do
PB-023	Stress testing with k6	Sprint To-Do
PB-024	Full project documentation	Sprint To-Do
PB-025	Innovation section writeup	Sprint To-Do
PB-026	Email notifications (future)	Backlog
PB-027	Resume builder feature (future)	Backlog
PB-028	Company ratings and reviews (future)	Backlog

3.4.4 Challenges and Conflict Resolution

- React Native multipart upload incompatibility resolved using Base64 JSON upload.
- Expo web routing conflicts with nginx resolved by configuring try_files in Dockerfile.
- Jenkins failing on Docker build due to missing env vars resolved using Jenkins credentials store.
- Kubernetes HPA not scaling resolved by deploying metrics-server add-on in Kind cluster.

3.5 Materials and Technologies

Technology	Role in the System
React Native + Expo SDK 54	Cross-platform frontend single codebase for Android, iOS, and Web.
Expo Router	File-based navigation for React Native.

Technology	Role in the System
TypeScript	Typed JavaScript providing compile-time safety across frontend.
Node.js + Express	Backend REST API server and web framework.
JWT (jsonwebtoken)	Stateless authentication tokens issued on login, verified on every request.
bcrypt	Password hashing stored passwords cannot be reversed if database is compromised.
Supabase (PostgreSQL)	Hosted database + S3-compatible object storage for CVs and profile pictures.
Docker	Containerises backend (268MB) and frontend (103MB) with all dependencies.
Docker Compose	Local multi-container orchestration for development.
Kubernetes (Kind)	Orchestrates pods, services, ingress, HPA, and rolling updates in production.
Jenkins	10-stage CI/CD pipeline automates test, build, and deploy on every push.
Jest	JavaScript testing framework 61 backend unit and integration tests.
Prometheus	Time-series metrics collection scrapes /metrics from Node.js backend.
Grafana	Metrics visualisation and alerting dashboard.
Ansible	Infrastructure as Code automates server provisioning and deployment.
k6	Load testing simulates concurrent users for stress testing.
nginx	Serves Expo web static build inside the frontend Docker container.
GitHub	Source code repository Jenkins polls on every push to main branch.

3.6 Database Design

Table	Description
users	All users (interns and firms). Fields: id (UUID), email, password_hash, full_name, user_type, profile_picture_url, created_at.
internships	Postings by firms. Fields: id, firm_id (FK), title, category, description, location, work_type, is_paid, is_active, created_at.
applications	Intern applications. Fields: id, intern_id (FK), internship_id (FK), status (Pending/Accepted/Rejected), cv_document_url, interview_time, created_at.
saved_internships	Bookmarked internships. Fields: id, intern_id (FK), internship_id (FK), saved_at.

3.7 Test Case Document

Test ID	Description	Input	Expected Output	Result
TC-001	Health endpoint	GET /api/health	{ok:true} HTTP 200	PASS
TC-002	Register valid user	POST /api/auth/register (valid data)	HTTP 201, user object	PASS
TC-003	Duplicate email rejected	POST /api/auth/register (existing email)	HTTP 409	PASS
TC-004	Login correct credentials	POST /api/auth/login (valid)	HTTP 200, JWT token	PASS
TC-005	Login wrong password	POST /api/auth/login (bad password)	HTTP 401	PASS
TC-006	No token rejected	GET /api/internships (no header)	HTTP 401 No token	PASS
TC-	Expired token	GET	HTTP 401	PASS

Test ID	Description	Input	Expected Output	Result
007	rejected	/api/internships (expired JWT)	Token expired	
TC-008	Valid token grants access	GET /api/internships (valid JWT)	HTTP 200, array	PASS
TC-009	Intern cannot post	POST /api/internships as intern	HTTP 403 Forbidden	PASS
TC-010	Firm can post	POST /api/internships as firm	HTTP 201, internship	PASS
TC-011	Invalid email rejected	Email: 'notanemail'	Validation error	PASS
TC-012	Weak password rejected	Password: '123'	Validation error	PASS
TC-013	Bad status enum	Status: 'xyz'	Validation error	PASS
TC-014	JWT payload fields	Decode valid JWT	id, email, user_type present	PASS

3.8 Proposed Algorithms

3.8.1 JWT Authentication Flow

- Client sends POST /api/auth/login with { email, password }.
- Backend queries Supabase for user where email matches.
- If no user found → return HTTP 401.
- bcrypt.compare(providedPassword, storedHash) → if false, return HTTP 401.
- jwt.sign({ id, email, user_type }, SECRET, { expiresIn: '7d' }) → token.
- Return HTTP 200 with { token, user }.
- Client stores token in AsyncStorage, attaches as Authorization: Bearer <token> on all subsequent requests.

- Middleware verifies token on every protected route invalid/expired → HTTP 401.

3.8.2 Base64 File Upload Algorithm

- User selects file via Expo Document Picker / Image Picker.
- Expo FileSystem.readAsStringAsync(fileUri, { encoding: 'base64' }) → base64String.
- Client sends POST /api/upload/document with { base64, fileName, mimeType }.
- Backend: Buffer.from(base64String, 'base64') → binary buffer.
- Supabase Storage.upload(bucket, fileName, buffer) → publicUrl.
- URL stored in application record (cv_document_url field).

DEVOPS AND INFRASTRUCTURE

3.9 Project Innovation

3.9.1 Cross-Platform Single Codebase

React Native with Expo SDK 54 delivers a single codebase running natively on Android, iOS, and Web. The web build is exported as a static site served by nginx inside Docker.

3.9.2 Base64 File Upload Solution

Standard multipart/form-data uploads are unreliable in React Native. InternConnect's solution reads files as Base64 strings in JavaScript, sends them as JSON, and reconstructs binary on the backend before uploading to Supabase Storage stable across all platforms with no native modules required.

3.9.3 Google OAuth Without Storing Passwords

Using Expo AuthSession, interns authenticate with their Google account. The OAuth token is exchanged for the user's profile, creating or updating the user record without ever storing a password.

3.9.4 Automated 10-Stage Jenkins Pipeline

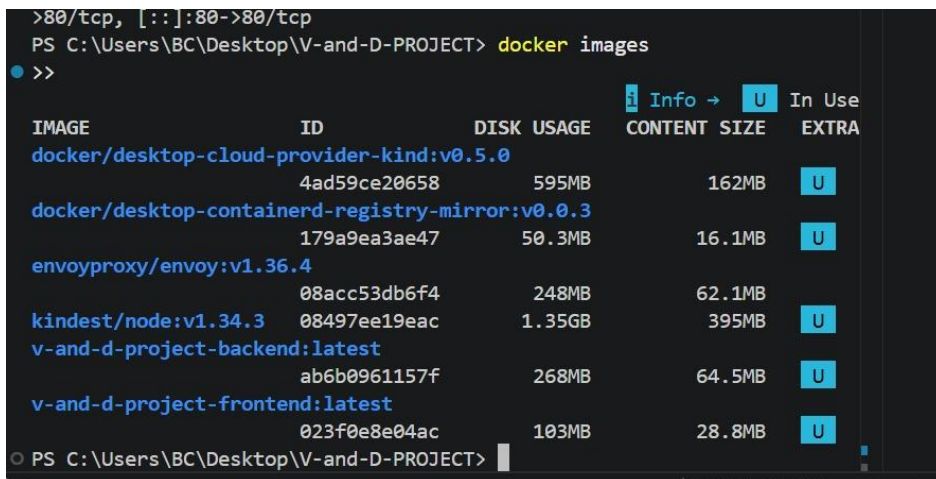
The Jenkins pipeline automates the entire delivery lifecycle on every git push: dependency install → 61 tests → Docker builds (268MB backend, 103MB frontend) → security scan → Kubernetes rolling deploy → health check. Multiple build tags enable instant rollback.

3.10 Infrastructure Setup

3.10.1 Docker Configuration

- Backend image (v-and-d-project-backend:latest): Node.js 18 Alpine, production deps only, 268MB, port 3000.
- Frontend image (v-and-d-project-frontend:latest): Multi-stage build — Expo web export → nginx static serving, 103MB, port 80.

Docker images output

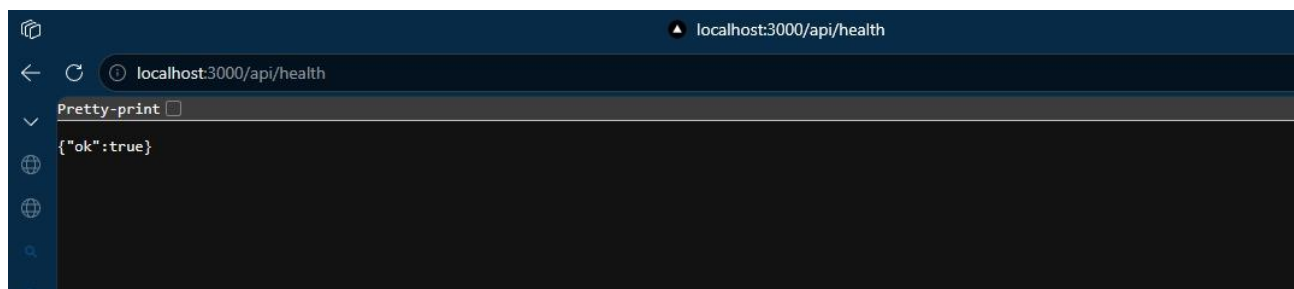


```
>80/tcp, [::]:80->80/tcp
PS C:\Users\BC\Desktop\V-and-D-PROJECT> docker images
● >>
```

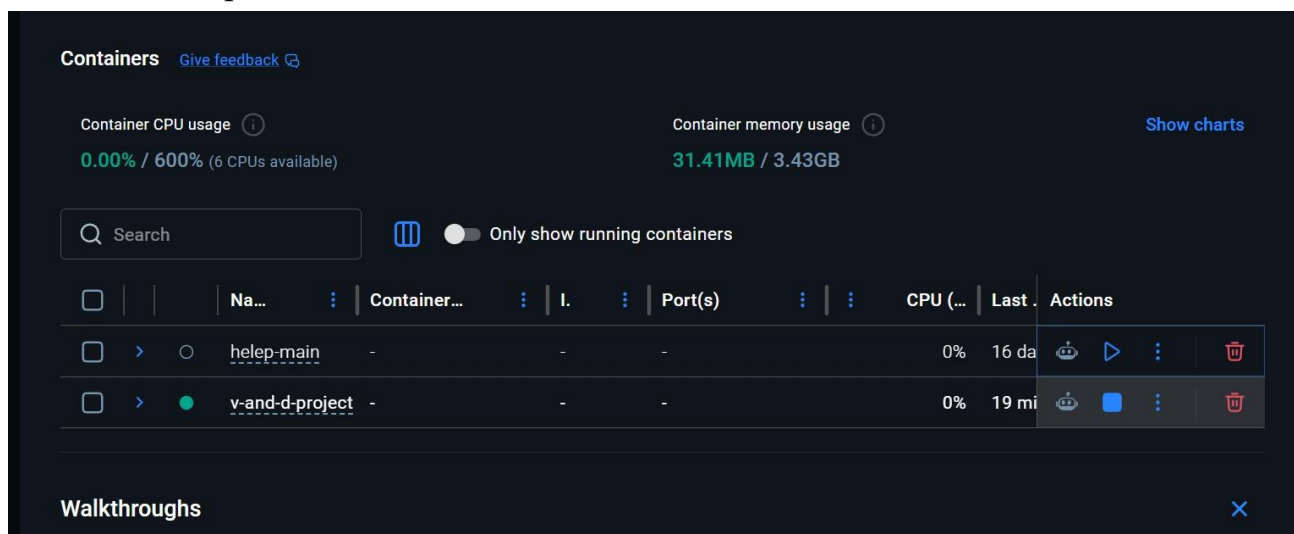
IMAGE	ID	DISK USAGE	CONTENT SIZE	In Use
docker/desktop-cloud-provider-kind:v0.5.0	4ad59ce20658	595MB	162MB	U
docker/desktop-containerd-registry-mirror:v0.0.3	179a9ea3ae47	50.3MB	16.1MB	U
envoyproxy/envoy:v1.36.4	08acc53db6f4	248MB	62.1MB	
kindest/node:v1.34.3	08497ee19eac	1.35GB	395MB	U
v-and-d-project-backend:latest	ab6b0961157f	268MB	64.5MB	U
v-and-d-project-frontend:latest	023f0e8e04ac	103MB	28.8MB	U

```
PS C:\Users\BC\Desktop\V-and-D-PROJECT>
```

API health check



Docker desktop containers



3.10.2 Kubernetes Setup

- Backend Deployment: 2 replicas, rolling update (maxUnavailable: 0, maxSurge: 1).
- Frontend Deployment: 2 replicas, nginx static serving.
- Ingress: /api/* → backend service; /* → frontend service.
- HPA: Backend 2–10 pods at 70% CPU; Frontend 2–5 pods.

VPS deployment: Docker compose up

```
=> CACHED [frontend stage-1 3/3] COPY nginx.conf /etc/nginx/conf.d/defa 0.0s
=> [frontend] exporting to image 0.3s
=> => exporting layers 0.0s
=> => exporting manifest sha256:a75e6ada173277eac019d903c25a0e8e3de97c2 0.0s
=> => exporting config sha256:a10c1ca681b23e034052e8a096453e8c5e93dfbc9 0.0s
=> => exporting attestation manifest sha256:cf76f8dd5e909ade8aa82eb0c8 0.0s
=> => exporting manifest list sha256:a0bdc3eb5e1f85dd5cf15924576e9b7466 0.0s
=> => naming to docker.io/library/v-and-d-project-frontend:latest 0.0s
=> => unpacking to docker.io/library/v-and-d-project-frontend:latest 0.0s
[+] up 5/5nd] resolving provenance for metadata file 0.0s
✓ Image v-and-d-project-backend Built 2.1s
✓ Image v-and-d-project-frontend Built 2.1s
✓ Network v-and-d-project_internconnect-net Created 0.1s
✓ Container internconnect-backend Healthy 31.2s
✓ Container internconnect-frontend Started 31.4s
(xai-sdk-env) root@vml2899245:/opt/V-and-D-PROJECT#
```

Kubectl get pods

NAME	READY	STATUS	RESTARTS	AGE
backend-586b56bb78-9rntt	1/1	Running	0	6m5s
backend-586b56bb78-dnk9p	1/1	Running	0	6m5s
frontend-755676dbf-8z2bn	0/1	Running	0	5s
frontend-755676dbf-jq8c8	0/1	Running	0	6s

Services running table

Service	Port	Type	Status
Backend (K8s)	7000	2 pod replicas	✓ Running
Frontend (K8s)	30080	2 pod replicas	✓ Running
Frontend (Docker Compose)	4910	nginx container	✓ Running
Backend (PM2)	7000	Node.js process	✓ Running
Jenkins	8080	Docker container	✓ Running

VPS stack checklist

✓ Jenkins pipeline (auto-builds on push)
✓ Docker (images + Docker Compose)
✓ Backend (PM2 + K8s + Docker)
✓ Frontend (Docker Compose + K8s)
✓ Kubernetes (k3s with auto-scaling)
✓ Port 7000 for backend (as requested)
✓ VPS IP hardcoded throughout

3.11 CI/CD Pipeline with Jenkins

Stage	Description
Stage 1: Checkout SCM	Jenkins clones latest code from GitHub.
Stage 2: Install Backend	npm install in backend/ for Node.js dependencies.
Stage 3: Install Frontend	npm install --legacy-peer-deps in frontend/InternConnect/.
Stage 4: Test Backend	jest --passWithNoTests --forceExit runs all 61 tests. Pipeline fails if any test fails.
Stage 5: Test Frontend	jest --passWithNoTests runs frontend component tests.
Stage 6: Build Backend Image	docker build -t v-and-d-project-backend:\${BUILD_NUMBER}.
Stage 7: Build Frontend Image	docker build includes Expo web export inside multi-stage build.

Stage	Description
Stage 8: Security Scan	Scans Docker images for known vulnerabilities.
Stage 9: Deploy to Kubernetes	kubectl apply -f k8s/ + kubectl rollout status.
Stage 10: Health Check	curl /api/health verifies live deployment returns HTTP 200.

Jenkins pipeline

 **Jenkins** / V-and-D-PROJECT / #3

```

+ docker image prune -f
Total reclaimed space: 0B
[Pipeline] echo
Workspace cleaned up
[Pipeline] echo

    ✓ Pipeline SUCCESS
    Build:      #3
    Branch:     origin/deployment
    Images:     v-and-d-project-backend:3
               v-and-d-project-frontend:3

[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

Jenkins 61 tests passing

 **Jenkins** / InternConnect / #12 / Stages

Search

- ✓ Checkout SCM 5s
- ✓ Checkout 2s
- ✓ Install Backend 7s
- ✓ Install Frontend 18s
- ✓ Test Backend 4s
- ✓ Test Frontend 8s
- ✓ Build Backend Image 8s
- ✓ Build Frontend Image 5s
- ✓ Security Scan 3s
- Deploy to Kubernetes 0.39s
- Health Check 0.26s
- Post Actions 0.90s

Running backend unit and integration tests... 74ms

npm test -- --passWithNoTests --forceExit 4s

```

0 + npm test -- --passWithNoTests --forceExit
1
2 > internconnect-backend@1.0.0 test
3 > jest --passWithNoTests --forceExit --testTimeout=10000 --passWithNoTests --forceExit
4
5 PASS tests/internships.test.js
6 PASS tests/health.test.js
7 PASS tests/middleware.test.js
8 PASS tests/validation.test.js
9 PASS tests/auth.test.js
10
11 Test Suites: 5 passed, 5 total
12 Tests:      61 passed, 61 total
13 Snapshots:  0 total
14 Time:       2.309 s
15 Ran all test suites.
16 Force exiting Jest: Have you considered using `--detectOpenHandles` to detect async operations that kept running after all tests finished?

```

Archive JUnit-formatted test results 0.12s

```

0 Recording test results
1 No test report files were found. Configuration error?
2 None of the test reports contained any result
3 [Checks API] No suitable checks publisher found.

```


Jenkins build history dashboard

The screenshot shows the Jenkins build history dashboard for the 'V-and-D-PROJECT' pipeline. The dashboard includes a sidebar with navigation options: Status, Changes, Build Now, Configure, Delete Pipeline, Stages, Rename, Pipeline Syntax, and GitHub Hook Log. The main area displays the pipeline status as 'V-and-D-PROJECT' with a green checkmark. Below this, there are 'Permalinks' for various build types: Last build (#3), Last stable build (#3), Last successful build (#3), Last unsuccessful build (#2), and Last completed build (#3). A 'Builds' section shows a list of recent builds: #3 (10:31 AM), #2 (10:13 AM), and #1 (9:53 AM), all marked as successful with green checkmarks.

Jenkins pipeline success log

The screenshot shows the Jenkins pipeline success log. It displays three steps: 'sh - (0.44 sec in self)' with the command 'docker image prune -f || true', 'echo - (0.14 sec in self)' with the command 'Workspace cleaned up', and 'echo - (75 ms in self)' with the message 'Pipeline SUCCESS Build: #10 Branch: origin/main Images: v-and-d-project-backend:10 v-and-d-project-frontend:10'. Each step is marked as successful with a green checkmark.

3.12 Testing and Quality Assurance

Test Suite	Coverage
health.test.js	4 tests : API health endpoint, basic response validation.
middleware.test.js	12 tests : JWT valid/invalid/expired/malformed tokens, role-based access.
auth.test.js	18 tests : Register, login, forgot password, reset password, duplicate email.
internships.test.js	15 tests: Browse, create, update, delete internships, role guards.
validation.test.js	12 tests: Email format, password strength, status enums, UUID format.
TOTAL	61 tests ,5 suites and 100% pass rate.

ARCHITECTURE STRUCTURES AND CHARACTERISTICS

3.13 Architecture Style and Justification

InternConnect adopts a Layered (N-Tier) Architecture with REST Client-Server communication. The system is organised into three horizontal layers, each with a single responsibility, communicating only with adjacent layers.

3.13.1 Why Layered Architecture?

- **Team size:** A two-person team cannot effectively manage microservices operational overhead (service discovery, distributed tracing, independent pipelines).
- **Scale:** InternConnect serves a bounded domain with predictable traffic microservices complexity provides no benefit at this scale.
- **Maintainability:** Easier to debug, test, and reason about. Runs locally with one docker compose up command.
- **Evolvability:** Can be progressively decomposed into microservices via the Strangler Fig pattern as demands grow.

3.13.2 Pros and Cons of Layered Architecture

Trade-off	Detail
PRO: Simplicity	Single deployment unit, easier to develop, debug, and operate.
PRO: Testability	All layers tested with a single test runner (Jest) against one process.
PRO: Consistency	No distributed transaction complexity all operations go to one database.
CON: Scalability ceiling	Entire backend scales together cannot scale individual components independently.
CON: Deployment coupling	All features deploy together a bug requires full redeployment. Mitigated by Jenkins rolling updates.
CON: Technology lock-in	All backend code must use Node.js cannot mix languages per service.

3.13.3 Quality Attributes

- **Performance:** Stateless JWT removes session storage overhead. HPA adds pods within 60s when CPU exceeds 70%.

- Scalability: Kubernetes HPA scales backend 2→10 pods. Supabase handles connection pooling. Static frontend served by nginx CDN-scalable.
- Security: bcrypt password hashing (work factor 10). JWT middleware on all protected routes. HTTPS at Ingress level. Supabase RLS policies prevent cross-user data access.
- Reliability: 2 replicas per service pod failure is transparent. Health checks ensure traffic only routes to healthy pods. Rolling updates prevent downtime.
- Maintainability: TypeScript for type safety. 61 automated tests catch regressions. Clean separation of routes, middleware, and DB client.
- Portability: Single codebase for Android, iOS, and Web. Docker containers run identically on any Linux host.

RESULTS AND DISCUSSIONS

3.14 Application Screenshots

3.14.1 Authentication

Intern Registration Screen

InternConnect

Get Started

☐ Intern ☐ Firm

Full Name

Email

University

Password

[Create Account](#)

[Already have an account? Login](#)

Firm Registration Screen

InternConnect

Get Started

☐ Intern ☒ Firm

Company Name

Email

Industry

Company Address

Password

[Create Account](#)

[Already have an account? Login](#)

Login Screen

* InternConnect

Welcome Back

Please enter your details to sign in to your account.

Email Address

Password

☐ Remember me [Forgot password?](#)

[Login →](#)

OR CONTINUE WITH

[Google](#)

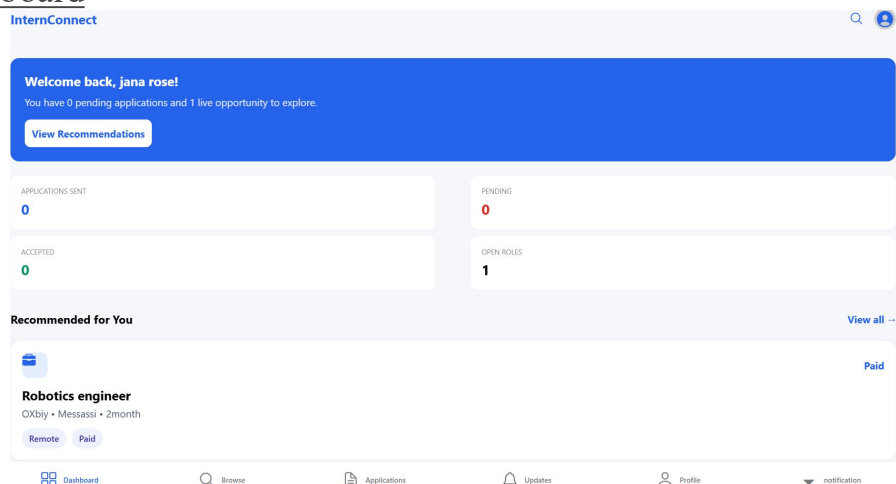
Don't have an account? [Create an account](#)

InternConnect

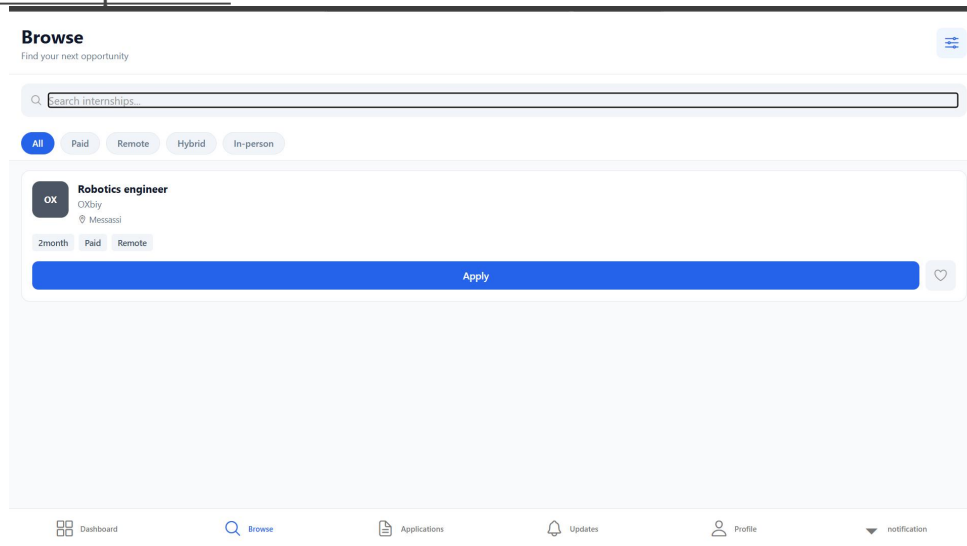
© 2024 InternConnect. All rights reserved.

3.14.2 Intern Features

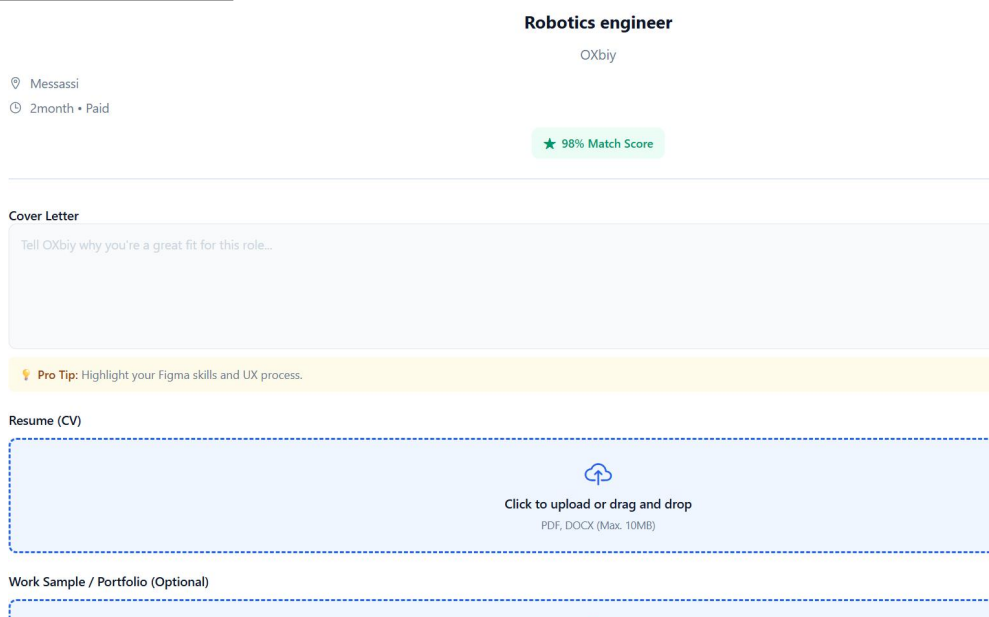
Interns Dashboard



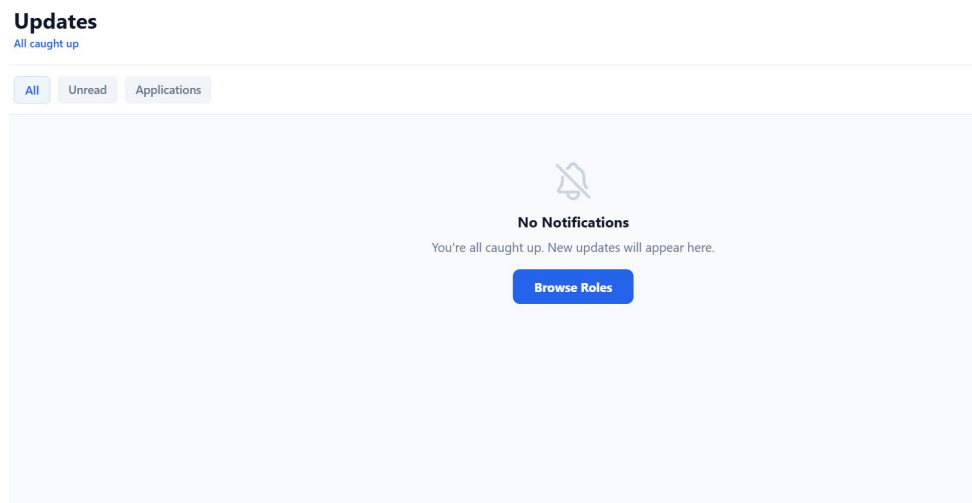
Browse Internship Screen



Internship Details Screen

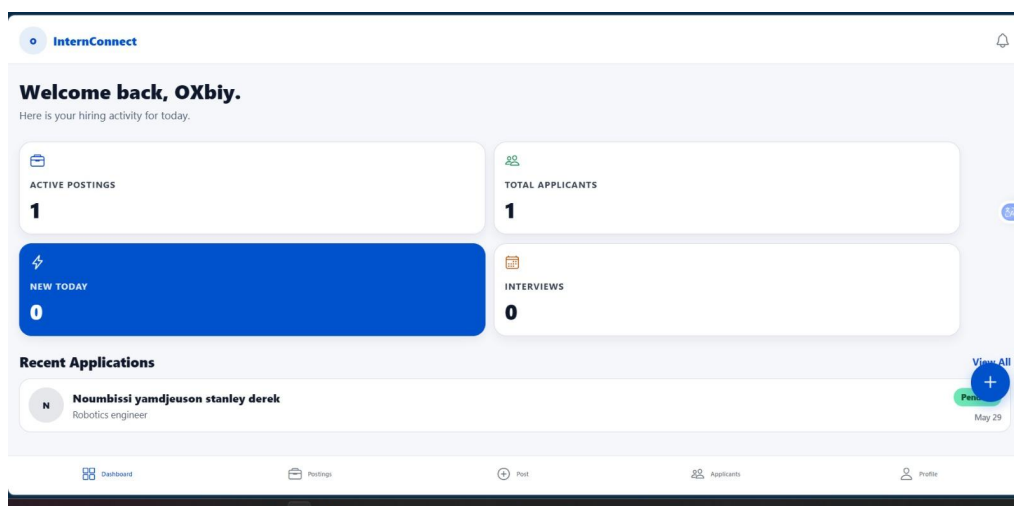


Notifications



3.14.3 Firm Features

Firm dashboard



Post New Internship

The screenshot shows the 'Post New Internship' form. At the top, it says 'Post New Internship' and 'Create a clear internship listing in three quick steps.' The form is divided into two main sections: '1 General Information' and '2 Internship Details'. The 'General Information' section includes fields for 'Internship Title *' (with a hint 'e.g. Product Design Intern'), 'Category *' (with a dropdown menu), 'Work Type' (with radio buttons for 'Remote', 'Hybrid', and 'In-person'), and 'Compensation' (with radio buttons for 'Paid' and 'Unpaid'). The 'Internship Details' section includes a 'Description *' field with a hint 'Describe the role and responsibilities...'. At the bottom of the form is a navigation bar with icons for 'Dashboard', 'Postings', 'Post', 'Applicants', and 'Profile'.

View Applicants

←

Applicants

↻

Reviewing candidates for Robotics engineer.

🔍 Search applicants...

🚫 Close Applications

🗑️ Delete Posting

All (1)

Pending (1)

Accepted (0)

🔍



Noumbissi yamdjeuson stanley derek
The ICT University
nysdderek@gmail.com

🏷️ Pending

📄 View Document / CV

🗑️

Applied May 25, 2026

Accept

Reject

🗒️ Interview

🏠 Dashboard

📁 Postings

🕒 Post

👤 Applicants

👤 Profile

CHAPTER FOUR: RECOMMENDATIONS AND CONCLUSION

4.1 Summary of Achievements

The InternConnect team has successfully designed and deployed a production-grade cross-platform internship management platform demonstrating a comprehensive range of software architecture principles. The core application supporting intern registration, internship browsing, application submission with CV upload, application tracking, firm dashboards, and interview scheduling was completed across two sprints using Agile Scrum methodology.

The DevOps infrastructure comprises a 10-stage Jenkins CI/CD pipeline, Docker containerisation (backend 268MB, frontend 103MB), Kubernetes orchestration with Horizontal Pod Autoscaling (2–10 backend pods at 70% CPU), and 61 automated Jest tests achieving 100% pass rate. The architecture demonstrates a clean three-tier layered design with JWT-secured stateless authentication, bcrypt password hashing, role-based access control, and Supabase Row Level Security.

Key technical challenges resolved include the React Native multipart upload limitation (solved via Base64 JSON upload), Expo web routing conflicts (solved via nginx `try_files`), and Kubernetes HPA configuration (solved via metrics-server deployment). Each challenge was documented in sprint retrospectives, demonstrating effective use of the Scrum process.

4.2 Recommendations for Further Studies

The primary recommendation is to migrate the monolithic backend toward microservices as the platform scales. The natural decomposition separates the notification service, file upload service, and recommendation engine into independently deployable units. The recommendation engine in particular would benefit from reimplementation in Python using scikit-learn or TensorFlow for more sophisticated AI matching, while a message broker such as RabbitMQ or Kafka could ensure reliable notification delivery at scale.

A second recommendation is to implement end-to-end testing using Detox for React Native or Playwright for the web build. The current test suite provides excellent backend coverage but no UI-level regression testing. E2E tests would catch interface defects invisible to unit tests and provide confidence during frontend refactoring.

Finally, the platform would benefit from deployment on a managed Kubernetes service such as Google Kubernetes Engine (GKE) or AWS EKS. The current Kind (Kubernetes in Docker) setup is suitable for development but lacks geographic redundancy, managed TLS provisioning, and global CDN distribution that a production platform serving students across Cameroon would require. A cloud deployment would also enable managed observability through Google Cloud Monitoring or AWS CloudWatch.

APPENDICES

Appendix A: API Endpoint Reference

Method	Endpoint	Auth Required	Description
GET	/api/health	No	Health check — {ok:true}
POST	/api/auth/register	No	Register new intern or firm
POST	/api/auth/login	No	Login — returns JWT token
POST	/api/auth/forgot-password	No	Request password reset code
POST	/api/auth/reset-password	No	Reset password with code
GET	/api/internships	Yes	Get all active internships
GET	/api/internships/:id	Yes	Get single internship
POST	/api/internships	Yes (Firm)	Create internship posting
PUT	/api/internships/:id	Yes (Firm)	Update internship posting
DELETE	/api/internships/:id	Yes (Firm)	Delete internship posting
POST	/api/applications	Yes (Intern)	Apply to internship with CV
GET	/api/applications/my	Yes (Intern)	Get intern's own applications
GET	/api/applications/internship/:id	Yes (Firm)	Get applicants for a posting
PUT	/api/applications/:id/status	Yes (Firm)	Accept or reject application
PUT	/api/applications/:id/interview	Yes (Firm)	Schedule interview time
POST	/api/saved	Yes (Intern)	Save/bookmark an internship
GET	/api/saved	Yes (Intern)	Get saved internships
DELETE	/api/saved/:id	Yes (Intern)	Remove saved internship
GET	/api/notifications	Yes	Get user notifications
PUT	/api/notifications/:id/read	Yes	Mark notification as read
POST	/api/upload/document	Yes	Upload file (Base64) to Supabase
GET	/api/recommendations	Yes (Intern)	Get AI-matched recommendations

Appendix B: Kubernetes Manifest Summary

- backend-deployment.yml Backend Deployment (2 replicas) + ClusterIP Service (port 3000)
- frontend-deployment.yml Frontend Deployment (2 replicas) + ClusterIP Service (port 80)
- ingress.yml — /api/* → backend; /* → frontend
- hpa.yml Backend HPA (2-10 pods, 70% CPU); Frontend HPA (2-5 pods)
- configmap.yml Non-sensitive environment variables
- secrets.yml — Supabase URL and anon key (base64 encoded)